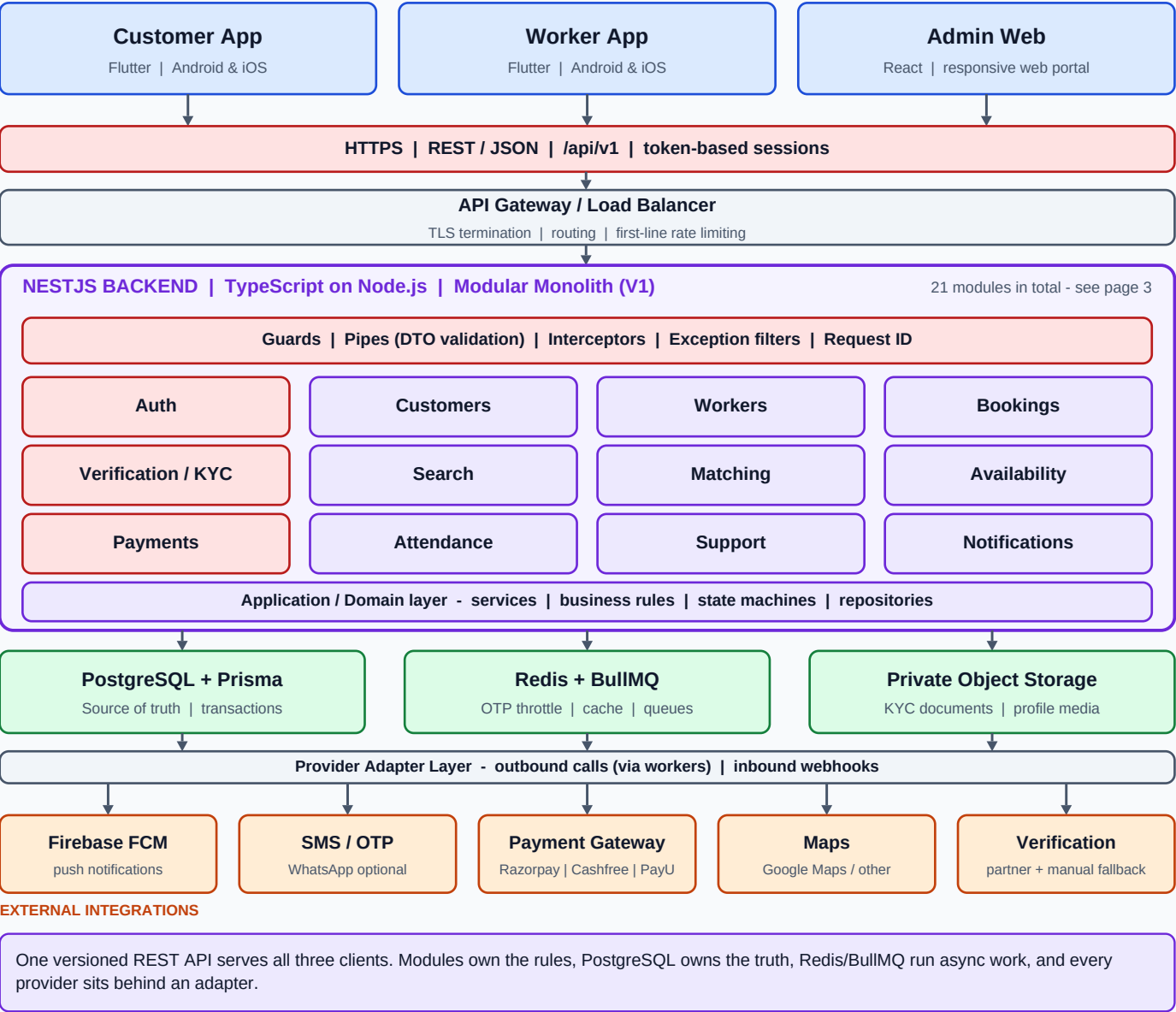


SEVANEST — BACKEND ARCHITECTURE

Production Backend Architecture for Customer, Worker and Admin Platforms

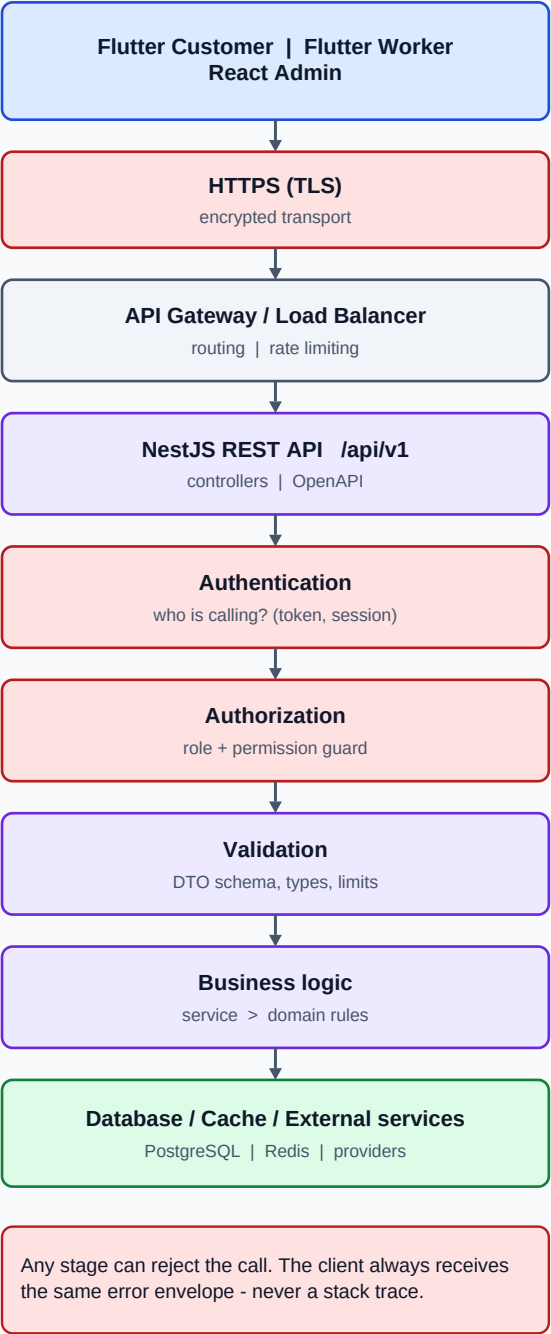


THE ARCHITECTURE IN SIMPLE TERMS

- Clients.** The Customer App, Worker App (Flutter) and Admin Web (React) never touch the database. They call one REST API over HTTPS.
- Gateway.** Terminates TLS, balances load and applies first-line rate limits.
- NestJS backend.** Authenticates, authorises (RBAC), validates input, then runs business rules inside self-contained modules.
- Data.** PostgreSQL is the only source of truth. Redis is temporary. KYC files sit in private object storage.
- Providers.** Payments, SMS/OTP, push, maps and verification are called through adapters, so a vendor can be swapped.
- Async work.** Notifications, reminders, reconciliation and reports run in background workers with retries.

DECISION STATUS

Decision	Status
NestJS + TypeScript	Recommended in this document
AWS vs Azure	Open - recommendation
Payment provider	Open - recommendation
Booking states	Open - recommendation
Retention policy	Open (SRS item 2)
Performance targets	Open (SRS item 7)



API CONVENTIONS

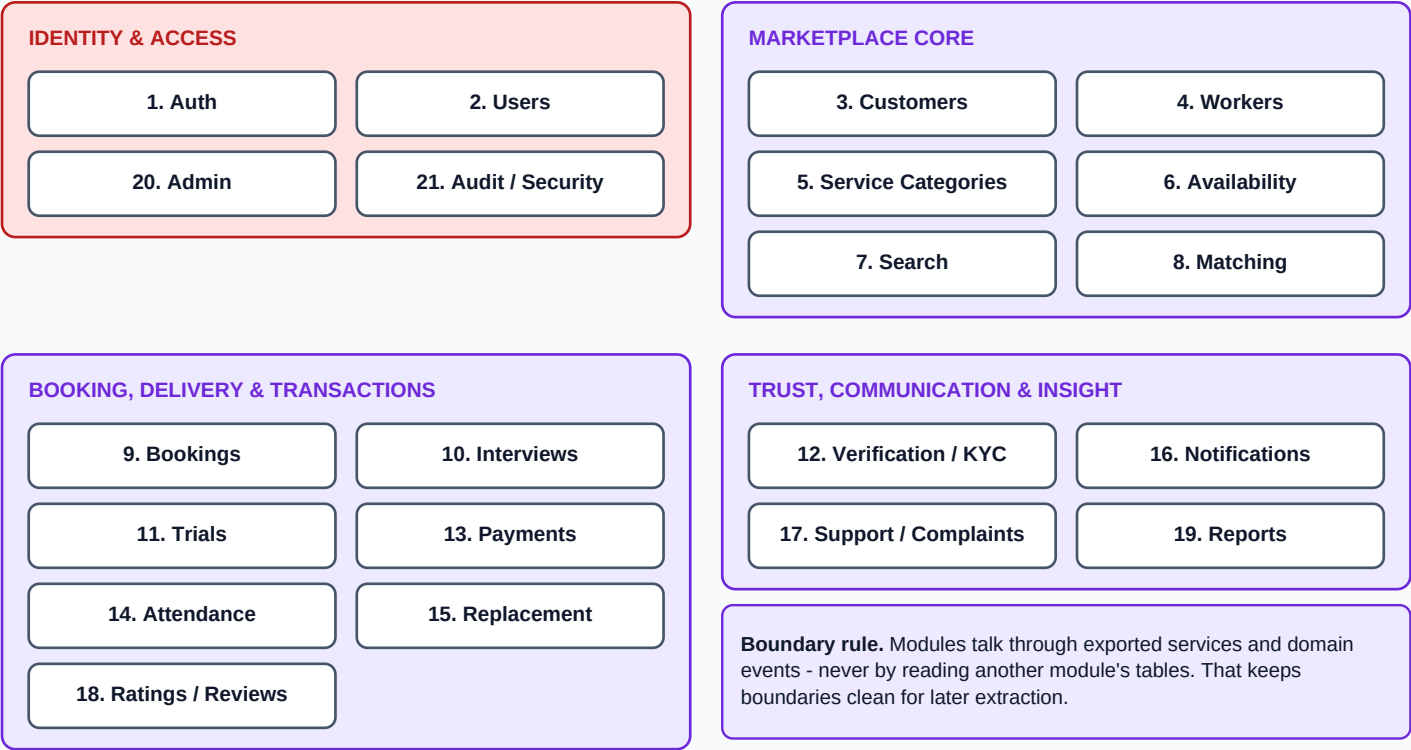
REST + JSON	Resource-oriented endpoints, JSON bodies, standard HTTP verbs and status codes.
HTTPS	TLS everywhere (NFR-SEC-001); no plain-HTTP listener in production.
Versioning	URI version prefix /api/v1; breaking changes ship as /api/v2 alongside v1.
Authentication	Authorization: Bearer access token; OTP login for customers and workers, credential login for admins.
Authorization	Route-level guards check role and permission on the server.
DTO validation	Every body, query and path parameter is validated against a typed DTO; unknown fields are stripped.
Error handling	One envelope: code, message, field details, requestId. Mapped from domain exceptions by a global filter.
Request IDs	X-Request-Id is generated at the edge and carried in logs, jobs and error responses.
Pagination	page + limit (or cursor) with a meta block: total, page, limit. Mandatory on search and lists (FR-WD-005).
Filtering	e.g. ?category=cook&area=madhapur&available=true
Sorting	e.g. ?sort=rating:desc
Idempotency	Idempotency-Key header on booking and payment creation to make retries safe.
API contract	OpenAPI / Swagger generated from the same DTOs; shared with Flutter and React teams.

EXAMPLE: POST /api/v1/bookings

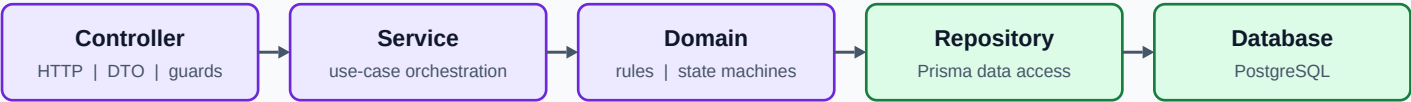
```
POST /api/v1/bookings
Authorization: Bearer <access_token>
Idempotency-Key: 7f3c-91ab-...
{ "workerId": "wrk_123",
  "categoryId": "cat_maid",
  "engagementType": "PART_TIME",
  "startDate": "2026-11-01",
  "addressId": "adr_456" }
```

What happens after it reaches the backend

- 1 Gateway terminates TLS, applies rate limit, stamps X-Request-Id.
- 2 AuthGuard validates the access token and loads the active session.
- 3 PermissionsGuard confirms booking.create for this role.
- 4 ValidationPipe checks the DTO (IDs, engagement type, dates).
- 5 BookingsService loads the worker: must be verified and available.
- 6 One DB transaction: insert booking, status history (REQUESTED), audit record.
- 7 After commit, a notification job is queued in BullMQ.
- 8 API returns 201 with the booking JSON and requestId.
- 9 A worker process sends the push / SMS to the worker app.



REQUEST LAYERING INSIDE EVERY MODULE



Controllers translate HTTP into service calls. Services orchestrate one use case inside a transaction. Domain code holds the rules (for example booking transitions). Repositories are the only code that talks to Prisma, so persistence can be tested and changed in one place.

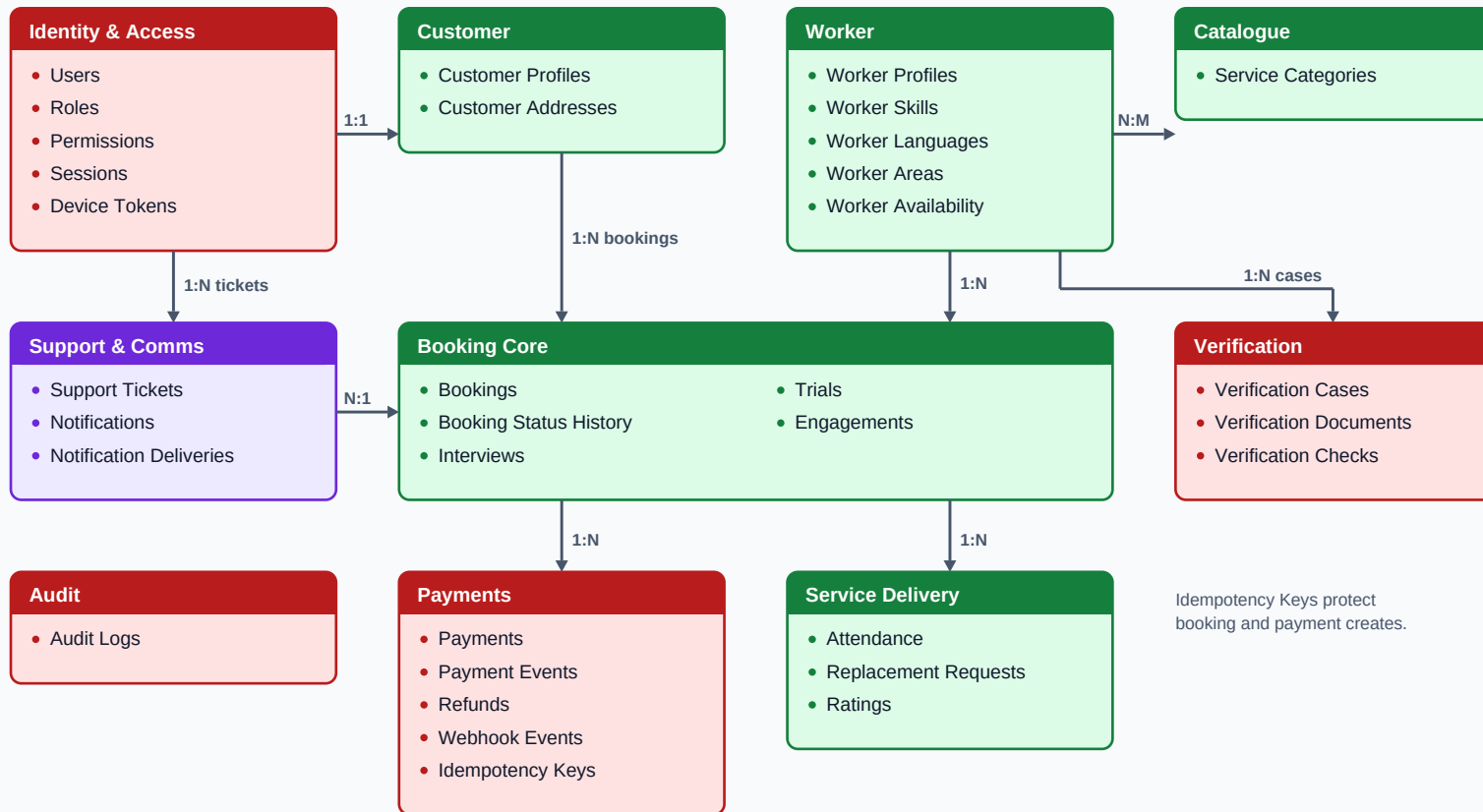
Deployment shape. V1 is deployed as one NestJS application with 21 internal modules. Each module has its own controller, service, domain and repository folders and a public interface. Workers (BullMQ consumers) run the same codebase as a second process, so background jobs reuse module logic without a second service.

WHY MODULAR MONOLITH, NOT MICROSERVICES

- **Faster development:** one codebase and one release fit the 16-week plan.
- **Easier deployment:** one container image, one pipeline.
- **Easier transactions:** booking + payment + history commit atomically in PostgreSQL.
- **Simpler debugging:** one trace, one log stream.
- **Lower infrastructure cost:** no service mesh or inter-service auth.
- **Clear boundaries:** enforced by Nest modules.
- **Future-ready:** any module can be extracted later.

RECOMMENDED FOLDER STRUCTURE

```
src/  
  common/  
  config/  
  database/  
  modules/  
    auth/  
    users/  
    customers/  
    workers/  
    categories/  
    availability/  
    search/  
    matching/  
    bookings/  
    interviews/  
    trials/  
    verification/  
    payments/  
    attendance/  
    replacements/  
    notifications/  
    support/  
    ratings/  
    reports/  
    admin/  
    audit/  
  integrations/  
  payments/  
  sms/  
  fcm/  
  maps/  
  storage/  
  verification/  
  jobs/  
  main.ts
```



DATABASE PRINCIPLES

- **PostgreSQL = source of truth.** Booking, payment, KYC metadata and audit state live here, never in Redis.
- **Prisma ORM.** Typed schema and queries shared with NestJS; one schema file describes all entities.
- **Migrations.** Versioned, reviewed SQL migrations run by CI/CD; schema and migration scripts are a V1 deliverable.
- **Transactions.** Multi-table changes (booking + history + audit, payment + booking) commit atomically.
- **Indexes.** Search filters (category, area, availability), status columns, provider references and foreign keys.
- **Foreign keys and constraints.** Referential integrity, unique provider event IDs, enum-style status checks.
- **Audit history.** Status-history tables and audit logs are append-only (DR-004, DR-005).
- **Backups.** Automated scheduled backups with point-in-time recovery on the managed service.

RELATIONSHIP TREE

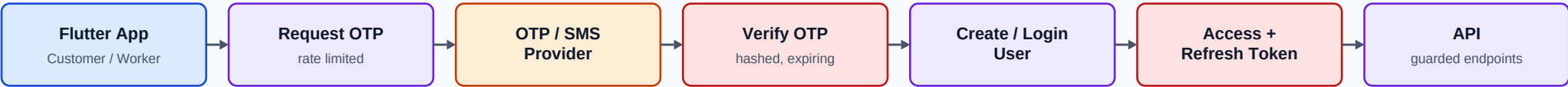
```
Customer
+-- Addresses
+-- Bookings
  +-- Worker (selected)
  +-- Booking Status History
  +-- Interview
  +-- Trial
  +-- Payment
  |   +-- Events, Refunds
  +-- Engagement
    +-- Attendance
    +-- Rating
    +-- Replacement Request
```

- User 1:1 Customer Profile or Worker Profile
- Worker Profile N:M Service Categories (via Worker Skills)
- Booking 1:N Status History (append-only), Interviews, Trials
- Booking 1:N Payments; Payment 1:N Events and Refunds
- Worker Profile 1:N Verification Cases > Documents, Checks
- Engagement 1:N Attendance; 0..1 Rating; 0..N Replacement Requests
- Audit Logs reference any entity: actor, action, record, timestamp

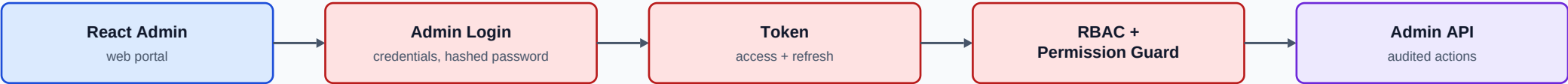
ARCHITECTURE RECOMMENDATION

Indicative model; final schema and migrations delivered per DR-003.

CUSTOMER / WORKER - mobile OTP login



ADMIN - Operations Admin, Super Admin, Verification Executive, Support Executive

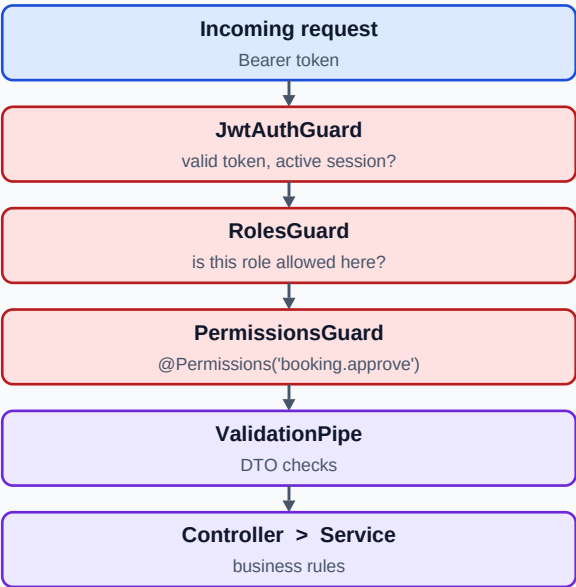


SESSION AND TOKEN DESIGN

ARCHITECTURE RECOMMENDATION

- **OTP.** Stored hashed with a short expiry and attempt cap in Redis; resend cooldown per mobile.
- **Access token.** Short-lived JWT carrying user, role and session ID (suggested ~15 min).
- **Refresh token.** Opaque random value, stored hashed in the sessions table and rotated on every use.
- **Session management.** One row per device session; admins can revoke.
- **Logout.** Revokes the session and deletes the device's FCM token.
- **Device tokens.** FCM tokens are bound to user and session so pushes stop after logout.
- **Rate limiting.** Redis counters per mobile, IP and device on OTP and API endpoints (FR-AUTH-003).
- **Brute-force protection.** Progressive lockout on repeated OTP or admin-login failures; passwords hashed with a slow algorithm (argon2 / bcrypt).

AUTHORIZATION CHAIN



EXAMPLE PERMISSION MATRIX

ARCHITECTURE RECOMMENDATION

Final matrix to confirm in design (SRS 3.18).

Permission	CUS	SA	OA	VE	SE
worker.verify		●	●	●	
worker.view		●	●	●	●
booking.create	●	●	●		
booking.approve		●	●		
payment.refund		●	●		
support.assign		●	●		●
report.view		●	●		
admin.manage_users		●			

CUS Customer SA Super Admin OA Operations Admin
VE Verification Executive SE Support Executive

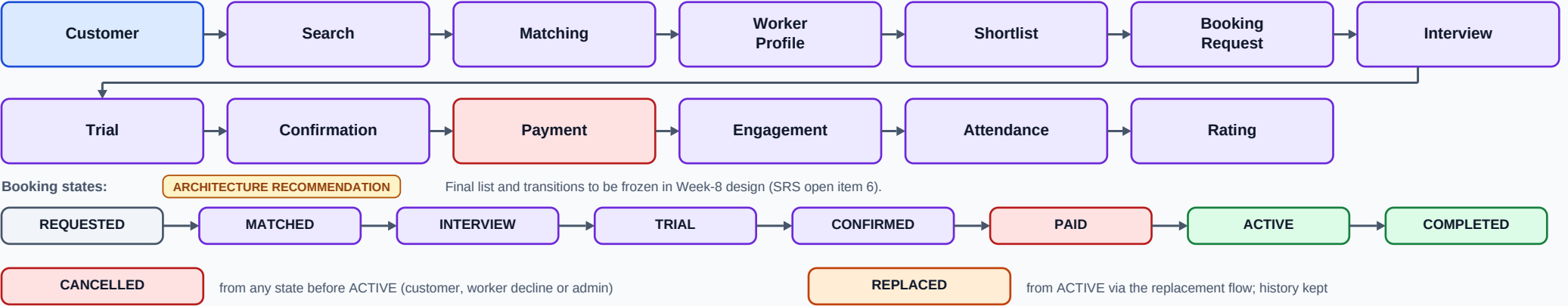
Frontend visibility is NOT security. Hiding a button in Flutter or React is not security. Every endpoint enforces authentication, role and permission on the server; the backend is the single authoritative decision point. Sensitive KYC access is further limited by role (FR-SEC-003) and every sensitive action is written to the audit log.

A | WORKER ONBOARDING

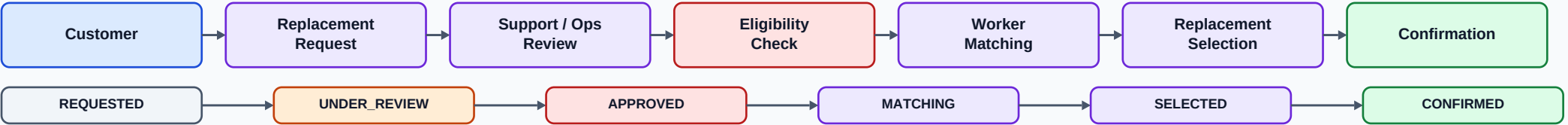


REJECTED and RECHECK_DUE branch from IN_REVIEW and APPROVED. If the verification partner is unavailable a manual status workflow takes over (FR-VER-011). A worker is shown as verified only after an authorised operations user approves (FR-VER-009).

B | CUSTOMER BOOKING



C | REPLACEMENT



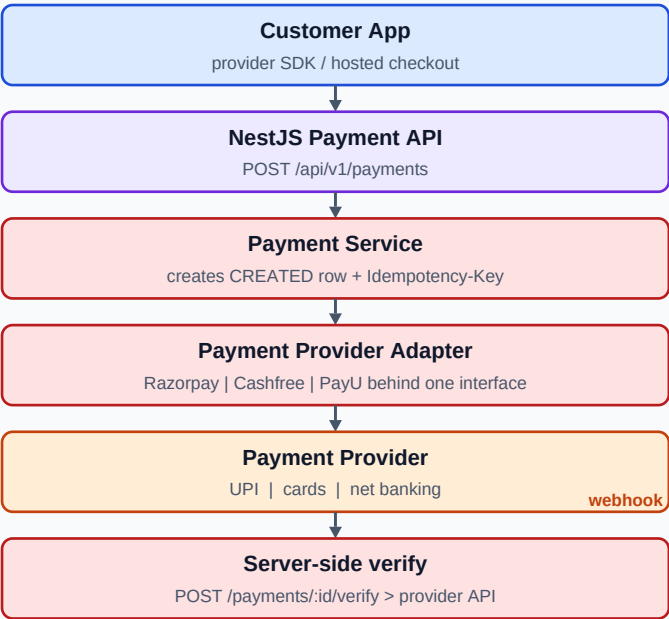
Engagement status: ACTIVE > REPLACEMENT_IN_PROGRESS > ENDED. The new worker receives a linked booking and the reason, approval and replacement booking are recorded (FR-REP-008). Eligibility rules for replacement and rating are an open SRS item (10).

Transition guard. A state-machine service rejects illegal moves with HTTP 409; only permitted booking states can be changed by admins (FR-BK-009).

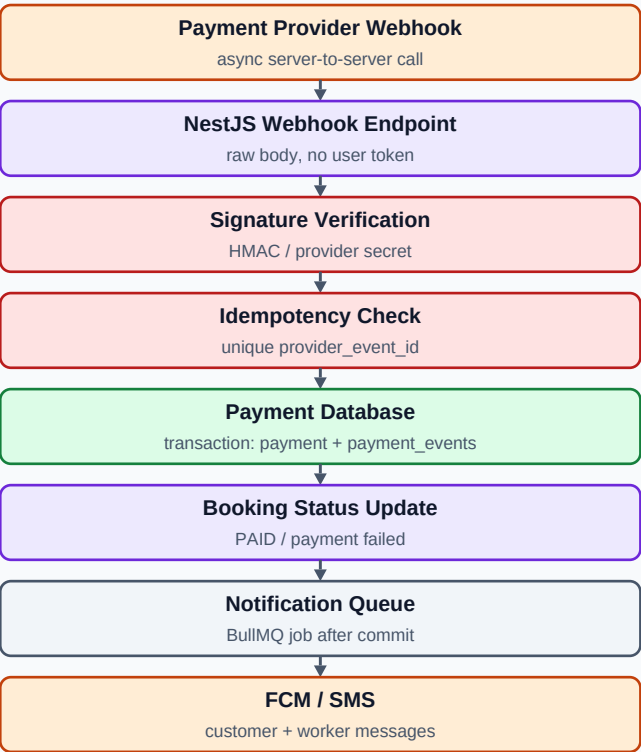
Atomic history. Every transition writes a status-history row and an audit record in the same database transaction (DR-004).

Notify after commit. Push / SMS jobs are queued only after the transition commits, so users are never told about a rolled-back change.

PHASE 1 | CHECKOUT



PHASE 2 | WEBHOOK (AUTHORITATIVE)

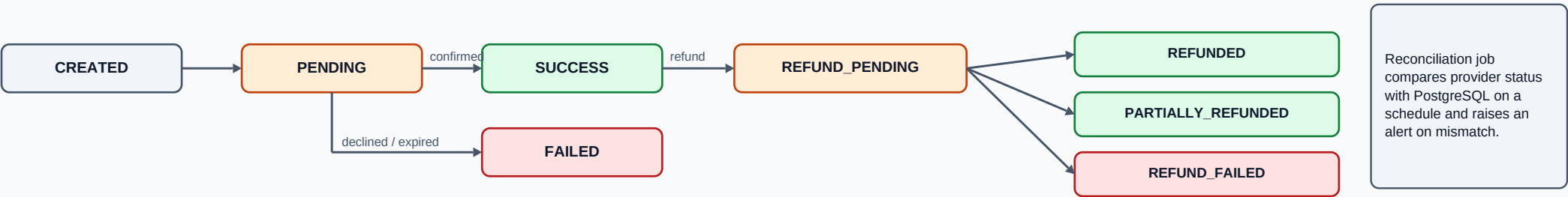


PAYMENT PRINCIPLES

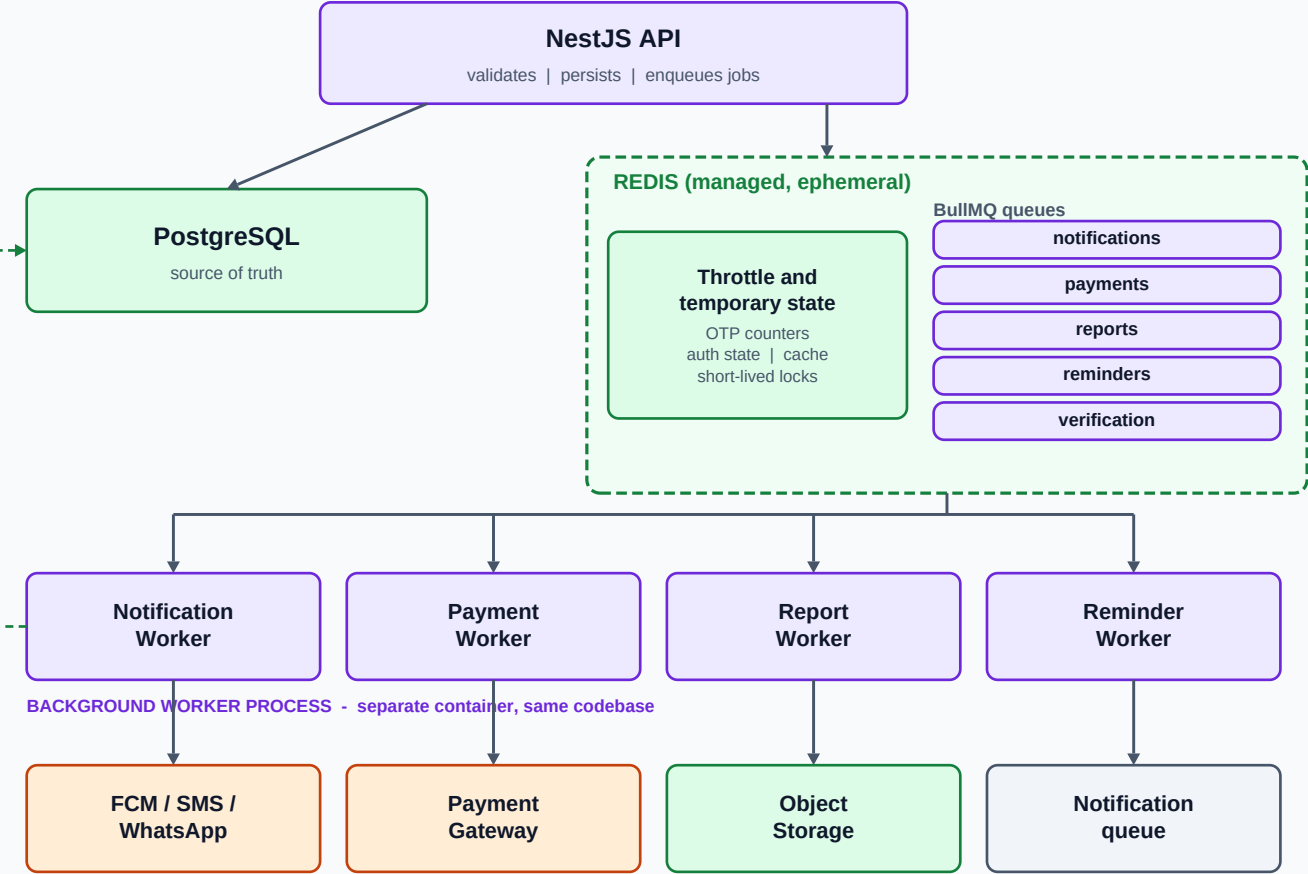
- **Never trust client success.** The app's "paid" result is only a hint; status changes only after server-side verification or a signed webhook.
- **Webhook signature.** Every webhook is verified against the provider secret before any state changes.
- **Idempotency.** Provider event IDs are unique in webhook_events; duplicate deliveries are acknowledged and ignored.
- **Audit trail.** payment_events is append-only; payment, refund and fee changes also write audit logs (FR-PAY-008).
- **Configurable fees.** Amounts come from admin-managed fee configuration, never hard-coded (FR-REV-003).
- **Refund handling.** Refunds need the payment.refund permission and are tracked to completion; status is visible to the customer (FR-PAY-004).
- **Reconciliation.** A scheduled job compares provider records with the database and flags mismatches.
- **Provider choice.** Razorpay, Cashfree or PayU is an open decision; the adapter makes it a configuration change.

PAYMENT STATE MACHINE

ARCHITECTURE RECOMMENDATION



Payment audit trail. Every transition is stored as a payment_events row (previous state, new state, source: API / webhook / job, timestamp) so the full payment history can be reproduced for support, finance and audit.



RETRIES WITH EXPONENTIAL BACKOFF

ARCHITECTURE RECOMMENDATION



Jobs retry a limited number of times with growing delays (plus jitter) so a provider outage is not hammered. After the last attempt the job moves to the failed set (dead-letter).

Dead-letter handling. Failed jobs keep payload and error, raise an alert, appear in the admin / ops view and can be replayed. Handlers are idempotent, so a replay never double-sends a message or double-processes a payment (EIR-SW-001, FR-NOT-006).

REDIS IS USED FOR

- OTP throttling and resend cooldowns
- Temporary authentication state (OTP codes, short-lived)
- Caching of read-heavy reference data
- BullMQ queues and background jobs
- Short-lived locks where appropriate (e.g. one reconciliation run at a time)

REDIS MUST NOT BE USED FOR

- Primary business database
- Authoritative booking state
- Authoritative payment state
- Permanent KYC records

BACKGROUND JOBS

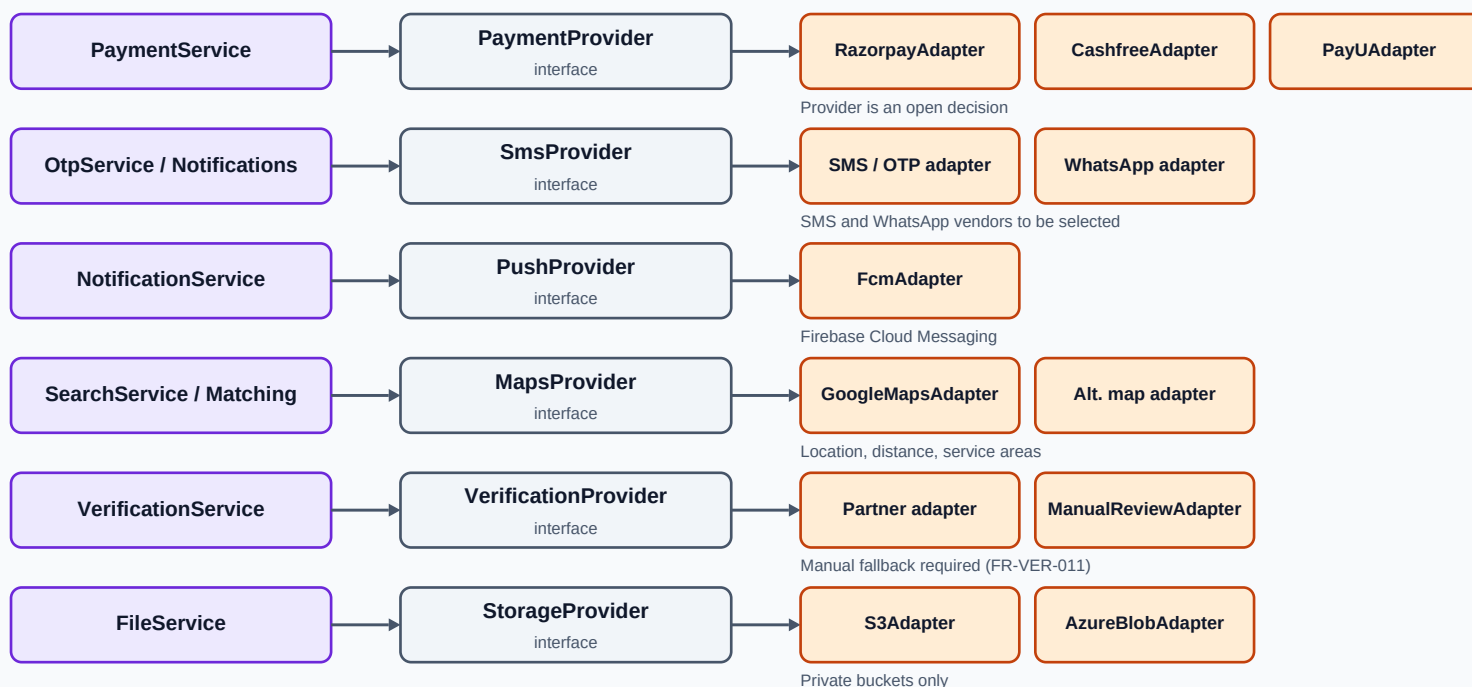
Notification sending	push / SMS / WhatsApp with retry
Interview reminders	scheduled before interview time
Booking reminders	trial, start-date, pending payment
Payment reconciliation	provider vs database
Report generation	heavy admin reports (FR-RPT)
Verification reminders	pending review, expiry / recheck
Retry failed provider calls	backoff, then failed set

RETRY POLICY

ARCHITECTURE RECOMMENDATION

- Up to ~5 attempts, base delay ~2 s doubling, with jitter
- Separate queues so a slow SMS vendor never blocks payments
- Queue depth, failure rate and age alerts

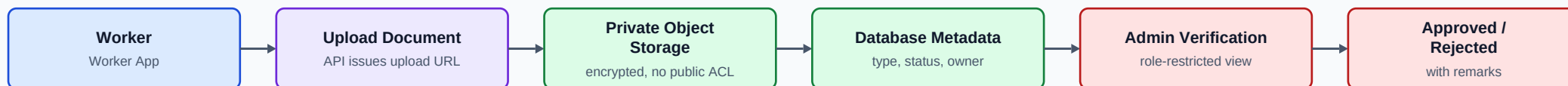
PROVIDER ABSTRACTION - DOMAIN SERVICE > INTERFACE > ADAPTERS



WHY ADAPTERS

- **Swap vendors** by configuration, not by rewriting business code (EIR-SW-002).
- **Sandbox early** for payment, maps and OTP (EIR-SW-003).
- **Fail gracefully**: timeouts, retries and failed-job handling (EIR-SW-001).
- **Testable**: business logic runs against fake providers.
- **Secrets** live in a secrets manager, never in code.

KYC DOCUMENT ARCHITECTURE



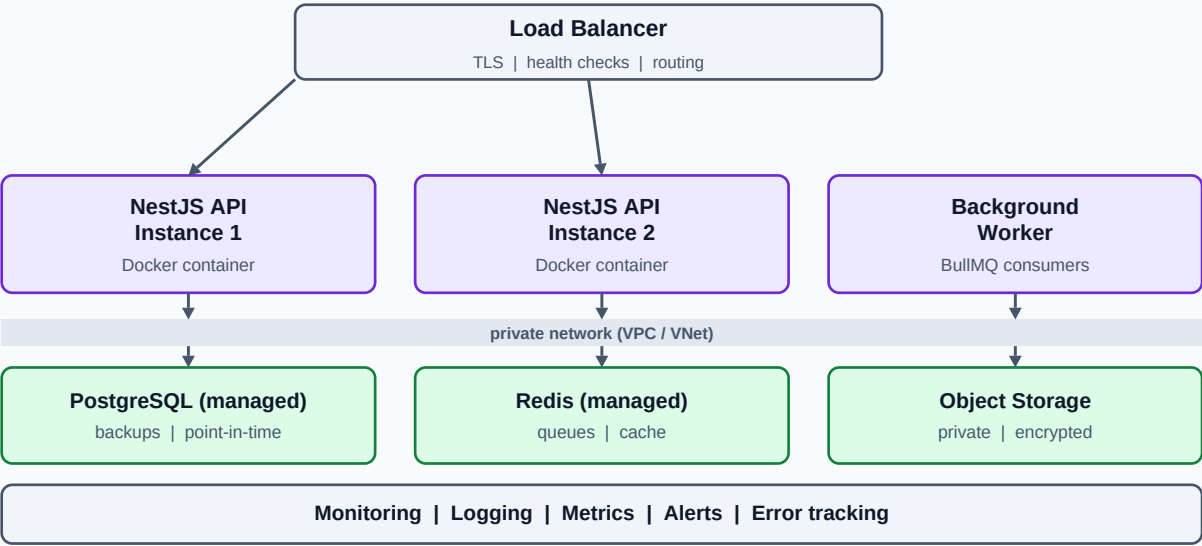
- **Private buckets**, no public KYC URLs (FR-SEC-004).
- **Signed URLs**: short-lived, issued per request after a permission check.
- **Restricted access**: only roles with document-view permission (FR-SEC-003).
- **Audit logs**: every view, download, approve and reject is recorded.

- **Encryption**: in transit (HTTPS) and at rest (storage + sensitive columns).
- **Document metadata**: type, status, reviewer, notes, expiry / recheck date.
- **Sensitive masking**: identity numbers masked on routine screens (FR-SEC-005).
- **Retention policy**: OPEN ITEM - to be defined before launch (SRS item 2); no period is assumed here.

Rule. Workers never receive another user's document links, and cards/profiles never expose full identity documents (FR-WD-007).



Separate environments (dev, staging, UAT, production) each with their own database, secrets and cloud resources.

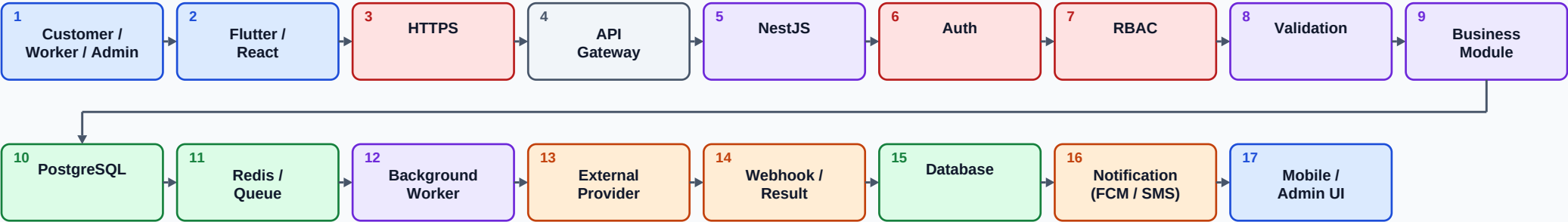


CLOUD AND OPERATIONS

AWS vs AZURE: OPEN

- **Docker + CI/CD:** one image per release, promoted through every environment.
- **Secrets management:** cloud secrets manager; no secrets in the repository.
- **Backups:** automated database backups and restore drills (NFR-OPS-003).
- **Logging and metrics:** structured JSON logs with request ID; CPU, latency, queue depth.
- **Alerts and health checks:** liveness / readiness probes, error-rate and failed-job alerts (NFR-OPS-002).
- **Error tracking:** exceptions grouped and linked to request ID.
- **Audit logs:** kept in the database for verification, payment and status changes.
- **Scaling:** add API instances horizontally; workers scale separately.

END-TO-END REQUEST FLOW - ONE COMPLETE PICTURE



Steps 1-9 are the synchronous path: the client gets a 2xx or a clear 4xx from the business module. Steps 10-17 are the asynchronous path: the module writes to PostgreSQL, queues work in Redis, a background worker calls the provider, the provider answers by webhook or result, the database is updated and the notification reaches the mobile or admin UI.

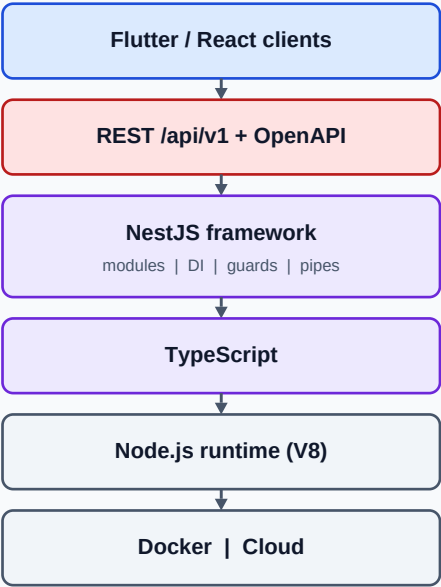
Node.js + Express vs NestJS vs Spring Boot

ARCHITECTURE RECOMMENDATION

Criterion	Node.js + Express	NestJS + TypeScript (selected)	Spring Boot
What it is	Minimal HTTP library on the Node.js runtime	Structured application framework running on Node.js	Java application framework on the JVM
Structure	No enforced structure; team conventions decide	Modules, providers and controllers enforce boundaries	Strong conventions through packages, beans and starters
Language	JavaScript or TypeScript (optional)	TypeScript-first; types shared with React Admin	Java or Kotlin
Dependency injection	Add-on libraries	Built-in DI container	Built-in, very mature
Guards / RBAC	Hand-written middleware	Guards and decorators for roles and permissions	Spring Security: powerful, mature
Validation and errors	Add-on libraries	Pipes, DTO validation, exception filters, interceptors	Bean Validation, exception handlers
OpenAPI	Manual or add-on	Generated from the same DTOs (@nestjs/swagger)	springdoc-openapi
Queues / jobs	Any library	First-class BullMQ and scheduling support	Capable (messaging, Spring Batch)
Fit for SevaNest V1	Fast start, but 21 modules risk inconsistent structure	Selected: integration-heavy platform, 16-week delivery, same language family as the web admin	Strong enterprise alternative; also technically capable of this scope

Spring Boot is a strong, proven enterprise option and the SRS lists it as a permitted choice. NestJS is selected for SevaNest on delivery requirements, frontend ecosystem and the integration-heavy architecture - not because Spring Boot is weaker.

THE STACK, LAYER BY LAYER



Node.js is the runtime. NestJS is the application framework on top of it.

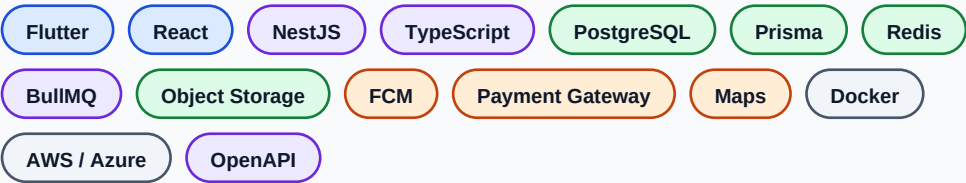
WHY NESTJS + TYPESCRIPT FOR SEVANEST

- **Modular architecture:** maps 1:1 to the 21 modules
 - **TypeScript:** one language with the React admin
 - **Dependency injection:** swap provider adapters easily
 - **Guards:** RBAC and permission checks per route
- **Validation:** DTO pipes on every input
 - **Interceptors:** logging, request ID, response shape
 - **Exception handling:** one error envelope for all clients
 - **OpenAPI:** contract shared with Flutter and React
- **Testing:** unit and e2e tooling built in
 - **Clean module boundaries:** ready for future extraction
 - **Integration ecosystem:** payments, FCM, S3, BullMQ, Prisma
 - **Faster V1 and teamwork:** consistent structure for 2 backend developers

Why not Express alone. Plain Express is not the main architecture: it would leave structure, DI, validation and guards to convention across a 21-module system.

ARCHITECTURE SUMMARY

TECHNOLOGY



Architecture style	MODULAR MONOLITH - V1
Primary database	POSTGRESQL
Primary API	REST /api/v1
Authentication	OTP + token sessions
Authorization	RBAC + permissions
Async	Redis + BullMQ
Storage	Private object storage
Payments	Provider adapter architecture
Deployment	Docker + cloud + CI/CD
Security	HTTPS + RBAC + audit + encryption + rate limiting
Scalability	Horizontal API scaling + background workers + managed database / cache

OPEN DECISIONS - NOT PRESENTED AS CLIENT-APPROVED

Decision	Status
Backend framework	NestJS + TypeScript recommended; Spring Boot also permitted by SRS 7.
Cloud provider	AWS or Azure - open (SRS item 9).
Payment provider	Razorpay, Cashfree or PayU - open; adapter keeps it swappable.
SMS / WhatsApp	Vendors to be selected.
Verification partner	Open (SRS item 5); manual fallback is built in.
Booking states	Recommended on page 6; frozen in Week-8 design (item 6).
Replacement / rating eligibility	Rules not defined (item 10).
Retention / deletion	Policy to be defined before launch (item 2).
Performance / SLA targets	No numeric targets in the PRD (item 7).
Role-permission matrix	Example on page 5; final per policy (item 11).

The SevaNest backend is designed as a secure, modular and production-ready platform that serves the Customer Mobile App, Worker Mobile App and Admin Web Portal through a single versioned API architecture while maintaining clear boundaries between business modules, data, asynchronous processing and external providers.